前置知識與本章目標

- 前置知識為第 1 章 ADT、不變量與成本模型，以及 C++ 物件生命週期。
- 能由索引域定義 Array ADT，推導多維索引的位址式，並說明連續儲存的限制。
- 能追蹤稀疏矩陣轉置及字串匹配，辨別「字元」與編碼單位。

Array ADT 的索引契約

- 一維陣列是固定索引域 $\{0,\ldots,n-1\}$ 到元素值的映射。`get(i)`、`set(i,x)` 的前提是 $0 \le i < n$。
- 長度改變是動態序列的額外能力，不是固定長度 Array ADT 的必要操作。
- `set(i,x)` 的後置條件為索引 `i` 的值變成 `x`，其餘索引的值不變。
- 連續記憶體是常見表示，不是 ADT 的數學定義。分段儲存也能提供索引語義，但成本可能不同。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

C++ 表示與生命週期

- `std::array<T,N>` 的長度編譯期固定。`std::vector<T>` 的長度可在執行期改變。
- 除特化 `vector<bool>` 外，`vector` 元素連續儲存。`size()` 和 `capacity()` 分別是現有元素數與已配置容量。
- 下標 `operator[]` 不能作為一般越界檢查工具。教材範例對外部索引先檢查，再存取。
- 容量重配會使原先指向元素的指標、參照與迭代器失效。`reserve` 不會把 `size` 增加到指定容量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

二維陣列的 row-major（逐列優先）位址

- 對 `R×C` 陣列，以固定寬度元素 `w` 位元組儲存，0 起始座標 `(i,j)` 的元素偏移為 `iC+j`。
- 若起點位址為 `b`，位址式是 `b+w(iC+j)`。此式只對同一連續區塊內的合法元素使用。
- 例子 `R=3,C=4,w=8`，座標 `(2,1)` 的元素偏移為 9，位元組偏移為 72。
- `vector<vector<T>>` 的各列可分別配置，不保證整個矩陣是一個連續區塊，不能直接套用上述整體位址式。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

逐列優先的座標與位址

先數完整列，再數該列內的欄位；位址與元素偏移的單位不同。

Row-major：從座標走到連續位址

矩陣 3×4；選取 (2,1)

(0,0)	(0,1)	(0,2)	(0,3)
(1,0)	(1,1)	(1,2)	(1,3)
(2,0)	**(2,1)**	(2,2)	(2,3)

先走過完整的 2 列：2×4
再往右 1 格：+1

offset = 2×4 + 1 = 9

連續元素槽位（數字是元素偏移）

0	1	2	3	4	5	6	7	8	9	10	11
									↑		

元素寬 8 bytes：位元組偏移 9×8=72；位址 b+72。

粗框與箭頭同時標示目標；此公式假設元素位於同一連續儲存區。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

多維步幅：row-major 與 column-major

對尺寸 $D_0 \times \cdots \times D_{d-1}$、合法索引 $0 \le i_t < D_t$，row-major 讓最後一個索引變化最快。第 t 維每增加 1，要跨過後面各維大小的乘積：

$$\text{offset}_{row} = \sum_{t=0}^{d-1} i_t \prod_{u=t+1}^{d-1} D_u.$$

Column-major 讓第一個索引變化最快，其步幅改為前面各維大小的乘積：

$$\text{offset}_{col} = \sum_{t=0}^{d-1} i_t \prod_{u=0}^{t-1} D_u.$$

空乘積定義為 1；元素寬 w 時再乘 w 才是位元組偏移。配置及位址運算前要檢查尺寸乘積與索引型別的範圍。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

索引次序與快取

- 在 row-major 儲存中，內圈沿欄索引 `j` 增加，會依序讀取相鄰元素。
- 若內圈沿列索引 `i` 增加，相鄰讀取的元素距離為 `C`，可能減少每個快取列的重用。
- 兩種走訪在 RAM 模型均為 `Θ(RC)`，實際時間仍可能不同。沒有實測前不報告快取速度比。
- 若 `R×C` 會溢位索引型別，必須在配置前檢查乘法範圍。合法的邏輯尺寸不代表機器可以配置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

稀疏矩陣的 COO 與 CSR

- `nnz=k` 表示非零項數。COO 用三元組 `(row,column,value)`，即列索引、欄索引與值，儲存，另保留矩陣尺寸。
- 本教材 COO 採 row-major 排序、索引合法、同一座標不重複、不存明確零。輸入若不符合，先排序、合併重複座標，再移除零值。
- CSR 用 `row_ptr`、`col_idx`、`values`。第 `i` 列的非零項位於 `[row_ptr[i],row_ptr[i+1])`。
- CSR 空間為 `O(R+k)`。對非常多空列的矩陣，`R` 的成本不能隱藏。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

稀疏矩陣例子

矩陣為 3×4。圖中的編號對應每個非零項；第 1 列的 CSR 區間是 $[1, 3)$，包含 -2 和 7。若 $x = [1, 2, 3, 4]^T$，則 $Ax = [10, 26, 8]^T$。



Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快速轉置依欄計數

1. 對原矩陣各欄計數，得到 `count[j]`。
2. 對 `count` 作前綴和，得到轉置後各列的起始位置 `start[j]`。
3. 依原 COO 順序走訪 `(i,j,v)`，把 `(j,i,v)` 寫入 `next[j]`，再增加 `next[j]`。
4. 原始 `start` 保留作為區間界線，不能被寫入游標破壞。

時間 `O(C+k)`，輔助空間 `O(C)`，輸出空間 `O(k)`。輸出若包含完整 CSR 列指標，還要保留 `C+1` 個位置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快速轉置為何不覆寫

- 原例各欄計數為 `[1,2,0,1]`，前綴起點為 `[0,1,3,3]`。
- 各欄的輸出區間互不相交，長度恰等於該欄的非零項數，因此每個輸入項有唯一輸出位置。
- 依輸入列順序穩定散佈，使轉置後同一列的欄號維持遞增。
- 結果為 `(0,1,-2)`、`(1,0,5)`、`(1,2,4)`、`(3,1,7)`。空欄自然對應空區間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

快速轉置：每次寫入的位置

對照前綴起點與游標，逐項核對輸出位置；空欄不佔任何輸出槽。

快速轉置：計數、分區與穩定散佈

count = [1, 2, 0, 1] start = [0, 1, 3, 3, 4]
next = [0, 1, 3, 3]（寫入前）

原 COO 項	原 column	寫入位置	轉置後項	更新後 next
(0,1,5)	1	1	(1,0,5)	[0,2,3,3]
(1,0,−2)	0	0	(0,1,−2)	[1,2,3,3]
(1,3,7)	3	3	(3,1,7)	[1,2,3,4]
(2,1,4)	1	2	(1,2,4)	[1,3,3,4]

輸出依位置 0 → 3：
(0,1,−2) (1,0,5) (1,2,4) (3,1,7)

第 2 欄 count=0，因此區間 [3,3) 為空；原矩陣列順序讓各區間穩定有序。
start 保留邊界；next 指向下一個可寫位置。

COO 快速轉置的完整配置與游標

```
transpose(R,C,entries):
    require sorted unique valid nonzero entries
    count=[0]*C
    for (r,c,v) in entries: count[c]+=1
    start=[0]*(C+1)
    for c=0..C-1: start[c+1]=start[c]+count[c]
    next=copy(start[0:C]); out=array(len(entries))
    for (r,c,v) in entries:
        out[next[c]]=(c,r,v); next[c]+=1
    return (C,R,out)
```

同一3×4例子的各項散佈為：i=0的(0,1,5)寫out[1]；i=1的(1,0,−2)寫out[0]；i=2的(1,3,7)寫out[3]；i=3的(2,1,4)寫out[2]。start=[0,1,3,3,4]，結束next=[1,3,3,4]，恰好等於各列右界。每個計數區間不交疊且寫入數等於長度，保證無覆寫與無遺漏；穩定按原row次序散佈，使轉置row內col有序。

空欄2的區間[3,3)長0；R=0,C=4時仍須回傳4×0尺寸與空項目。時間Θ(C+k)，輸出Θ(k)、輔助Θ(C)；若輸入未正規化，排序與座標合併成本必須另計。加法合併也需尺寸完全相同，不只是nnz相同。

稀疏加法與乘法成本

- 兩個已排序 COO 相加，可像合併排序般同時前進。相同座標相加，結果為零就不輸出。
- 輸入非零項數為 k_1,k_2 時，合併成本 $O(k_1+k_2)$，不含輸入正規化。
- 稀疏矩陣向量乘法 $y_i=\Sigma_j\ A_{ij}\ x_j$ 以 CSR 執行為 $O(R+k)$，含初始化輸出。
- 稀疏矩陣乘法成本不能只寫 $O(k_1+k_2)$。輸出可能變稠密，且累加同座標需要額外資料結構。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

稀疏加法與矩陣向量乘的完整操作

```
add_COO(A,B):
    require same dimensions and canonical row-major entries
    i=0; j=0; out=[]
    while i<len(A) and j<len(B):
        if coord(A[i])<coord(B[j]): out.append(A[i]); i+=1
        else if coord(B[j])<coord(A[i]): out.append(B[j]); j+=1
        else:
            value=A[i].value+B[j].value
            if value!=0: out.append((A[i].row,A[i].col,value))
            i+=1; j+=1
    append remaining A[i:] and B[j:]; return out
CSR_matvec(R,C,row_ptr,col_idx,values,x):
    require len(x)==C
    y=[0]*R
    for r=0..R-1:
        for p=row_ptr[r]..row_ptr[r+1]-1:
            y[r]+=values[p]*x[col_idx[p]]
    return y
```

稀疏加法與矩陣向量乘的完整操作 （續）

令B只有(0,1,−5),(2,0,3)，與本章A相加時，首座標(0,1)抵消；A的(1,0,−2),(1,3,7)先輸出；接著B的(2,0,3)再A的(2,1,4)，結果4項而非直接kA+kB=6項。矩陣向量乘x=[1,2,3,4]，row0累加5×2=10；row1依序−2×1=−2，再+7×4得26；row2為4×2=8。

維持每列區間端點0≤row_ptr[r]≤row_ptr[r+1]≤k，row_ptr[0]=0、row_ptr[R]=k且每個col_idx∈[0,C)。這些是CSR表示不變量，不能只驗三個array長度相同。算術成本假設固定範圍值，不替溢位或任意精度求和承諾常數時間。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

String ADT 與編碼層次

- 字串可定義為字母表 Σ 上的有限序列，基本操作有長度、連接、子字串與比較。
- `std::string` 儲存 `char` 編碼單位，可以含內嵌 `\0`。邏輯長度以 `size()` 為準，不能一律用 C 字串終止規則。
- UTF-8 中一個 Unicode 碼點可占多個位元組，一個使用者感知的字形又可能包含多個碼點。
- 本章匹配演算法預設比較固定大小的符號。若輸入採 UTF-8 位元組，回傳位置也是位元組位置。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

字串操作的複製與檢視語義

- 若用長度及連續編碼單位表示字串，讀取既有索引可為 O(1)，複製長度 k 的子字串需 Θ(k) 時間與輸出空間。
- 連接長度 N、M 的兩字串並產生獨立結果，需 Θ(N+M) 複製。反覆用新結果複製整個前綴，累積成本可能二次成長。
- 非擁有的字串 view 只保留起點與長度，可常數時間切片，但不延長原字串生命週期。來源被釋放或重配後，view 不能繼續使用。
- 字典序比較要逐符號找第一個差異，最壞成本包含共同前綴長度及結束判定，為 O(共同前綴長度+1)。雜湊值、指標相同或長度相同都不能一般地取代內容相等判定。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

直觀匹配與 KMP 前綴函數

- 文字長度 `N`、樣式長度 `M`。逐起點比對的符號比較次數最多為 `O(NM)`；加上初始化與空輸入處理，時間上界為 `O(1+NM)`，如大量重複字元在最後一位才失配。
- 定義 `π[i]` 為樣式 `P[0..i]` 的最長真前綴與後綴共同長度。
- 若目前已匹配 `q` 個字元卻失配，改以 `q=π[q-1]` 嘗試較短的已知邊界，不回退文字讀取位置。
- π 的建立和搜尋各以回退次數的總量分析，總時間 `O(N+M)`，輔助空間 `O(M)`。

回退鏈列出已匹配前綴的較短 border（同為前綴與後綴的區段）。失配後仍可能成立的匹配長度必須是該前綴的 border，因此略過其他長度不會漏解。每次成功延伸至多使 q 增加 1，每次回退嚴格減少 q；建立 π 與搜尋各自的回退總數，都由先前延伸次數支付，不會超過線性數量。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP 的原創追蹤

- 樣式 `P=ababaca`，前綴函數為 `[0,0,1,2,3,0,1]`。
- 已匹配 `ababa` 後遇到 `b`，樣式下一位期待 `c`。先從 `q=5` 退到 `π[4]=3`。
- 此時 `P[3]=b` 可匹配目前文字字元，得到 `q=4`，並未重新掃描已讀的文字。
- 在文字 `abababaca` 中，完整匹配的 0 起始位置是 2。空樣式在本教材定義為位置 0 的匹配，需在介面明訂。

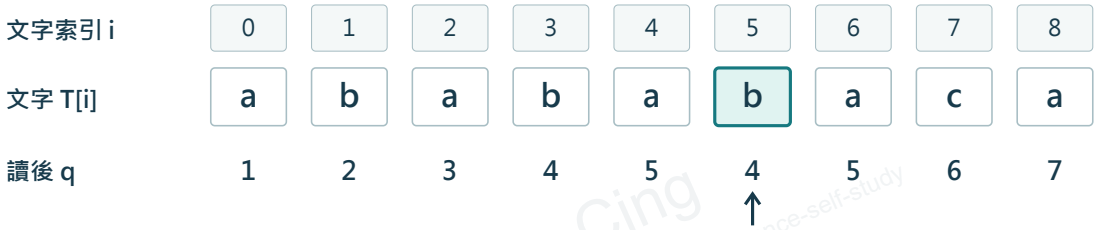
Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP：回退時沿用同一個文字字元

圖中的 q 是目前已匹配的樣式前綴長度。

KMP 追蹤：文字不回退，樣式沿 border 回退

P = ababaca π = [0, 0, 1, 2, 3, 0, 1]



i=5：原本 q=5，P[5]=c 與 T[5]=b 不同。

回退 q ← π[4] = 3；同一個 b 與 P[3] 相符，更新 q=4。

i=8 時 q=7；第一個匹配起點為 8−7+1=2。

q 是已匹配前綴長度；粗框與箭頭標記回退步驟。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

KMP 建表與搜尋的完整迴圈

```
prefix(P):
    pi=[0]*len(P)
    for i=1..len(P)-1:
        q=pi[i-1]
        while q>0 and P[q]!=P[i]: q=pi[q-1]
        if P[q]==P[i]: q+=1
        pi[i]=q
    return pi
first_match(T,P):
    if len(P)==0: return 0
    pi=prefix(P); q=0
    for i=0..len(T)-1:
        while q>0 and P[q]!=T[i]: q=pi[q-1]
        if P[q]==T[i]: q+=1
        if q==len(P): return i-len(P)+1
    return not_found
```

P=ababaca 建表各步為 i=1:b,q=0→0；i=2:a,0→1；i=3:b,1→2；i=4:a,2→3；i=5:c,3→1→0且不延伸；i=6:a,0→1，得[0,0,1,2,3,0,1]。在T=abababaca，讀每字後q為1,2,3,4,5,4,5,6,7；第5號文字b使q先5→3再增至4，完整匹配終於i=8，起點2。

KMP 建表與搜尋的完整迴圈 （續）

前綴函數建構的迴圈只在i≥1且P非空時取P[q]；搜尋先處理空樣式，且在q=M時立刻回傳，故不讀P[M]。若要列舉重疊匹配，成功後令q=pi[M−1]再繼續，而非直接清零。固定符號比較下總O(N+M)；UTF-8若以byte處理，位置仍是byte offset，不能暗稱字形索引。

參考資料與署名

Lin Cing，《資料結構》手寫筆記；C++ 標準草案 vector.overview 與 vector.capacity；Knuth, Morris, Pratt, Fast Pattern Matching in Strings (1977)。

Lin Cing 原創教學整理，原創內容以 CC BY 4.0 授權。

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study